

INTERNSHIP PROJECT REPORT

Create User Login System using AWS Cognito and LAMP Stack

Submitted By

Bhavani M G

June 2026

Contents

Abstract	2
Objective	3
Introduction	4
Literature Survey	5
Existing System	6
Proposed System	7
Tools and Technologies Used	8
System Architecture	8
Methodology	9
Source Code	14
Testing and Validation	14
Results and Output	15
AWS Cognito Security Features	17
Advantages	17
Challenges Faced	18
Future Enhancements	19
Conclusion	19
Learning Outcomes	19
References	20

ABSTRACT

The User Login System using AWS Cognito and LAMP Stack is a secure authentication solution developed using Amazon Cognito and Laravel. The project demonstrates cloud-based identity management, OAuth 2.0 authentication, secure user registration, sign-in, sign-out, password recovery, and token-based authorization. AWS Cognito provides a managed authentication service while Laravel serves as the application framework. The project eliminates the complexity of building authentication from scratch and provides a scalable, secure, and reliable user management solution. The implementation includes User Pool creation, App Client configuration, Hosted UI integration, Laravel configuration, and optional deployment using Amazon EC2 with LAMP, DNS, and SSL.

OBJECTIVE

1. Create an AWS Cognito User Pool.
2. Configure Cognito App Client.
3. Configure Cognito Hosted UI.
4. Integrate AWS Cognito with Laravel.
5. Implement secure login and logout.
6. Understand OAuth 2.0 authentication.
7. Verify user authentication workflow.
8. Deploy application using LAMP Stack.
9. Configure DNS and SSL.
10. Learn cloud identity management concepts.

INTRODUCTION

Authentication is one of the most important requirements in modern web applications. Organizations need secure systems that verify user identities and protect sensitive information. Traditional authentication systems require developers to manage password storage, hashing, account verification, session management, and security policies manually.

Amazon Cognito is a fully managed identity service provided by AWS that simplifies authentication and authorization. It enables developers to add sign-up, sign-in, and user management functionality without implementing complex authentication mechanisms. Laravel is a modern PHP framework widely used for developing secure web applications. Integrating Laravel with AWS Cognito provides a scalable and secure authentication solution.

The project focuses on configuring AWS Cognito User Pools, App Clients, Hosted UI, and integrating them with Laravel running on a LAMP environment.

Authentication has become a critical component of modern web applications due to the increasing need for secure access control and protection of user information. Organizations across various domains rely on authentication systems to ensure that only authorized users can access sensitive resources and services. As cyber threats continue to evolve, implementing secure authentication mechanisms has become a necessity rather than an option.

Cloud-based authentication services have emerged as a practical solution for reducing development complexity while improving security. AWS Cognito provides a fully managed authentication infrastructure that allows developers to focus on application development rather than user management and security implementation. By integrating Cognito with Laravel, organizations can achieve secure and scalable authentication without maintaining custom authentication servers.

LITERATURE SURVEY

Cloud identity management systems have become essential for secure application development. Research shows that managed authentication systems reduce vulnerabilities compared to custom-built authentication modules. AWS Cognito supports OAuth 2.0, OpenID Connect, and SAML standards, making it suitable for enterprise-grade authentication systems.

Benefits include:

- Reduced development complexity.
- Improved security.
- Automatic scalability.
- Built-in user verification.
- Password policy enforcement.
- Multi-factor authentication support.

Several studies have highlighted the importance of centralized identity management systems in reducing security vulnerabilities. Traditional authentication systems often suffer from weak password policies, improper session handling, and poor scalability. Cloud identity providers address these challenges through built-in security mechanisms and standardized authentication protocols.

AWS Cognito has gained popularity due to its support for OAuth 2.0, OpenID Connect, and SAML. Research indicates that applications using managed authentication services experience fewer authentication-related vulnerabilities and lower maintenance costs. These advantages make Cognito a preferred choice for modern web application development.

EXISTING SYSTEM

Traditional authentication systems store user credentials directly within application databases. Developers are responsible for designing and implementing authentication mechanisms such as user registration, password hashing, login validation, session management, and password recovery. These systems require continuous monitoring and maintenance to ensure security and reliability.

In many organizations, authentication modules are developed as part of the application itself. While this approach provides flexibility, it also increases development complexity and introduces potential security vulnerabilities if best practices are not followed. Developers must regularly update authentication logic to address emerging security threats and compliance requirements.

As the number of users increases, managing authentication infrastructure becomes more challenging. Organizations must ensure high availability, scalability, and secure handling of user credentials, which often requires additional resources and expertise.

Limitations:

- High maintenance due to continuous updates and monitoring.
- Security risks arising from improper implementation of authentication mechanisms.
- Poor scalability when handling large numbers of users.
- Complex implementation requiring significant development effort.
- Manual user management and account administration.
- Increased infrastructure and operational costs.
- Difficulty in implementing advanced features such as Multi-Factor Authentication (MFA).
- Higher risk of password-related vulnerabilities and data breaches.

PROPOSED SYSTEM

The proposed system uses AWS Cognito as the authentication provider and Laravel as the web application framework.

Components:

- AWS Cognito User Pool
- Cognito Hosted UI
- Laravel Application
- OAuth 2.0 Authentication Flow
- LAMP Environment

The proposed system separates authentication responsibilities from application logic. Instead of storing and validating user credentials within the Laravel application, authentication is delegated to AWS Cognito. This architecture improves security by reducing the attack surface and ensuring that sensitive authentication processes are handled by AWS infrastructure.

The integration of Cognito Hosted UI further simplifies the login process by providing a secure and customizable authentication interface. Users can register, verify their accounts, log in, and recover passwords using a standardized workflow managed entirely by AWS Cognito.

Advantages:

- Secure authentication.
- Cloud-managed identity management.
- Automatic scaling.
- Reduced operational overhead.
- Better user experience.

TOOLS AND TECHNOLOGIES USED

Technology	Purpose
AWS Cognito	Authentication Service
Laravel	Web Framework
PHP	Backend Development
Apache	Web Server
MySQL	Database
Linux	Operating System
AWS EC2	Hosting
Route53	DNS Management
SSL/TLS	Secure Communication
OAuth 2.0	Authentication Protocol

SYSTEM ARCHITECTURE

Authentication Workflow:

1. User accesses Laravel Application.
2. User clicks Login.
3. Application redirects to Cognito Hosted UI.
4. User enters credentials.
5. Cognito authenticates the user.
6. Authorization code is generated.
7. Laravel exchanges code for tokens.
8. User is redirected to dashboard.

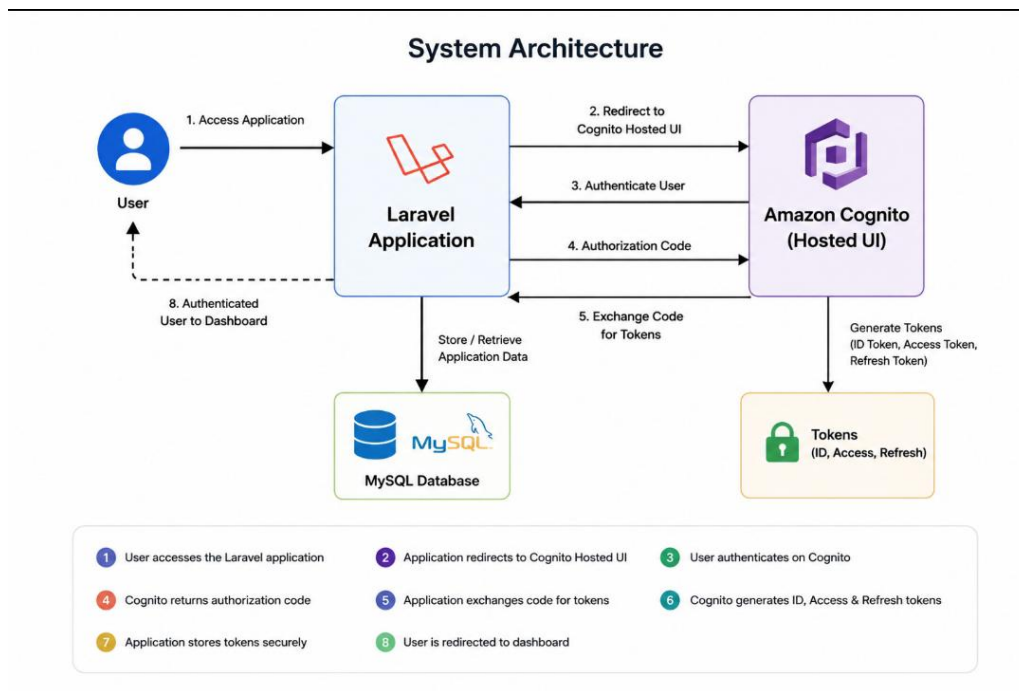


Figure 1: System Architecture

The architecture follows a client-server model where the Laravel application acts as the client application and AWS Cognito serves as the authentication provider. Communication between the application and Cognito occurs through secure HTTPS connections, ensuring confidentiality and integrity of user credentials and authentication tokens.

When a user initiates the login process, the application redirects the request to Cognito Hosted UI. After successful authentication, Cognito generates authorization tokens that are used to verify user identity and grant access to protected resources within the Laravel application.

METHODOLOGY

Task 1: Create AWS Cognito User Pool

Steps:

1. Login to AWS Console.
2. Open Amazon Cognito.
3. Create User Pool.
4. Configure Sign-In Options.
5. Configure Password Policy.

6. Configure Email Verification.

7. Create User Pool.

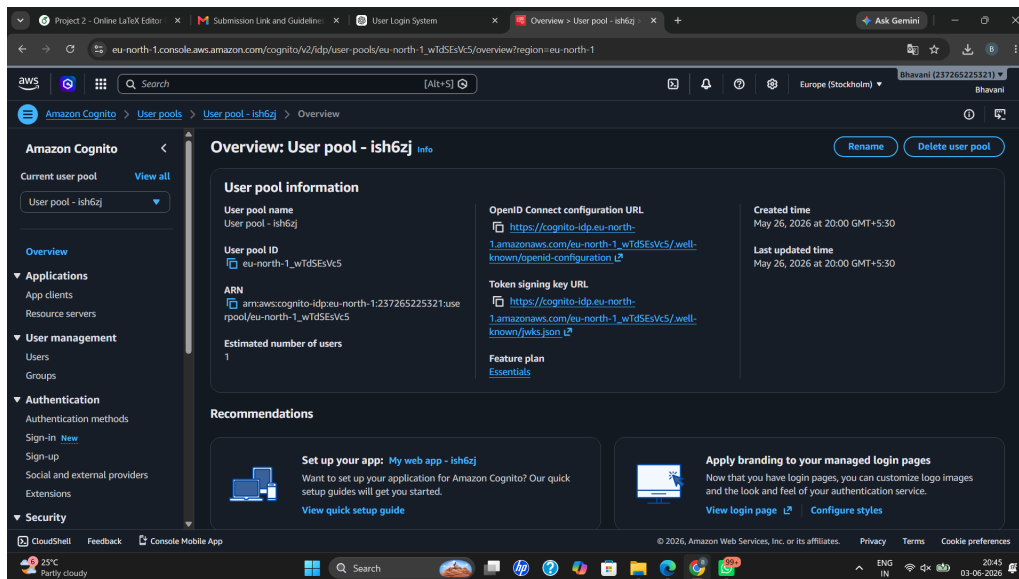


Figure 2: AWS Cognito User Pool

Task 2: Configure App Client

1. Create App Client.
2. Configure Callback URL.
3. Configure Logout URL.
4. Enable OAuth Flows.
5. Enable OpenID Scopes.

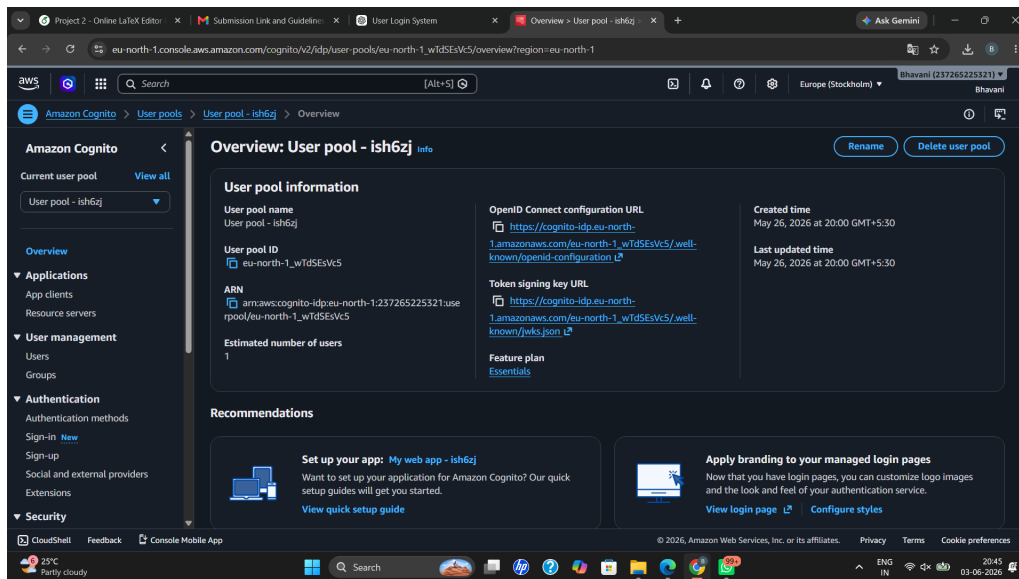


Figure 3: App Client Configuration

Task 3: Configure Laravel

Install Laravel:

```
composer create-project laravel/laravel cognito-app
```

Install dependencies and configure environment variables.

```
Windows PowerShell
- Installing phpunit/php-file-iterator (5.1.1): Extracting archive
- Installing theuser/tokenizer (1.3.1): Extracting archive
- Installing sebastian/lines-of-code (3.0.1): Extracting archive
- Installing sebastian/complexity (4.0.1): Extracting archive
- Installing sebastian/code-unit-reverse-lookup (4.0.1): Extracting archive
- Installing phpunit/php-code-coverage (11.0.12): Extracting archive
- Installing phar-io/version (3.2.1): Extracting archive
- Installing phar-io/manifest (2.0.4): Extracting archive
- Installing myclabs/deep-copy (1.13.4): Extracting archive
- Installing phpunit/phpunit (11.5.55): Extracting archive
Generating optimized autoload files
> Illuminate\Foundation\ComposerScripts::postAutoloadDump
> @php artisan package:discover --ansi

[INFO] Discovering packages.

laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE

81 packages you are using are looking for funding.
Use the 'composer fund' command to find out more!
PS C:\Users\bhava\OneDrive\Desktop\IP_2_CC\cognito-app> php -m | findstr zip
zip
PS C:\Users\bhava\OneDrive\Desktop\IP_2_CC\cognito-app> php artisan serve

[INFO] Server running on [http://127.0.0.1:8000].

Press Ctrl+C to stop the server

2026-06-03 23:44:47 / ..... " 17s
2026-06-03 23:45:04 / ..... " 2s
```

Figure 4: Laravel Configuration

Laravel was selected because of its clean architecture, extensive documentation, and built-in support for modern web development practices. Environment variables were configured

using the .env file to securely store Cognito credentials, callback URLs, and application-specific configuration parameters.

The framework provides middleware and routing capabilities that simplify the implementation of authentication workflows. Integration with Cognito enables Laravel to leverage cloud-managed authentication while maintaining flexibility in application development.

Task 4: Hosted UI Authentication

Hosted UI provides a secure login page managed by AWS Cognito.

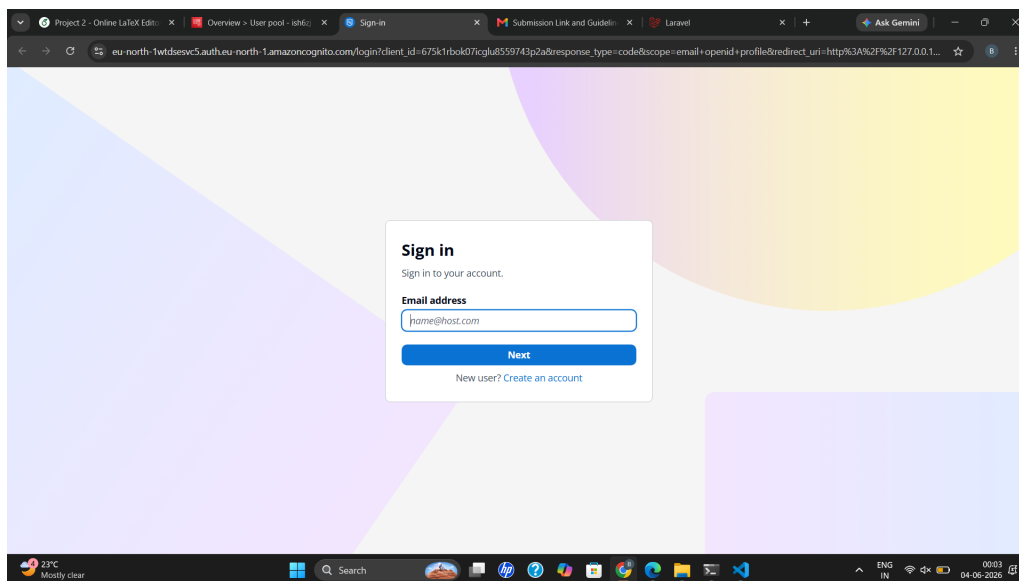


Figure 5: Hosted UI Login

Task 5: User Registration and Verification

The user creates an account using the Cognito Hosted UI. AWS Cognito sends a verification code to the registered email address to confirm the account before authentication.

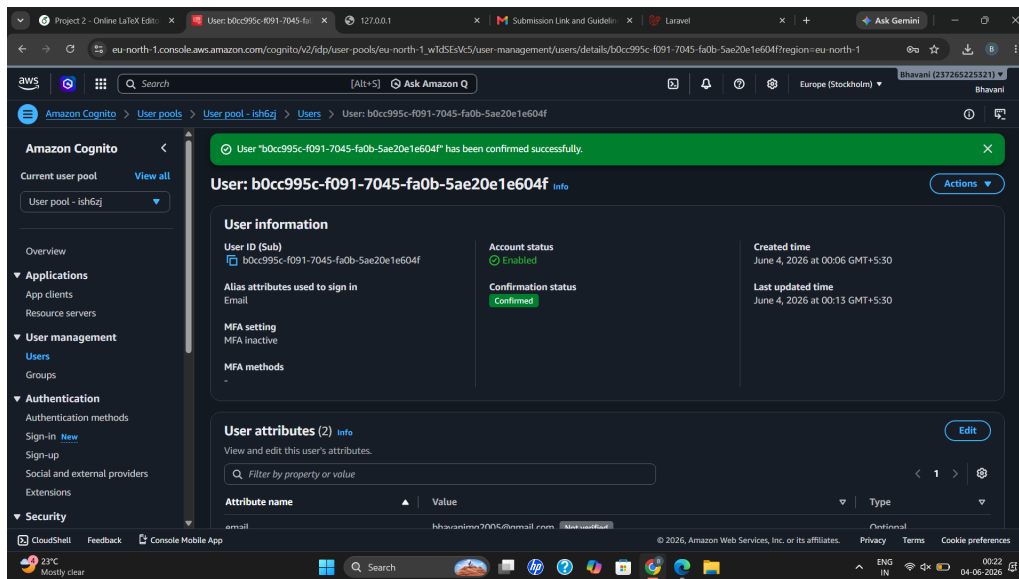


Figure 6: User Registration and Verification

Task 6: EC2 Deployment (Optional)

1. Launch EC2 Instance.
2. Install Apache.
3. Install PHP.
4. Install MySQL.
5. Deploy Laravel Application.
6. Configure DNS.
7. Configure SSL.

SOURCE CODE

Environment Variables

```
AWS_DEFAULT_REGION=ap-south-1
COGNITO_USER_POOL_ID=xxxxxxx
COGNITO_CLIENT_ID=xxxxxxx
COGNITO_CLIENT_SECRET=xxxxxxx
COGNITO_REDIRECT_URI=http://localhost:8000/callback
```

Routes

```
Route::get('/login',
[AuthController::class,'login']);

Route::get('/callback',
[AuthController::class,'callback']);

Route::get('/logout',
[AuthController::class,'logout']);
```

Controller Example

```
public function login()
{
    return redirect($this->cognitoUrl);
}
```

TESTING AND VALIDATION

Testing and validation were performed to ensure that all authentication workflows operated correctly and securely. Functional testing was conducted for user registration, email verification, login, logout, and password recovery processes.

The callback URL configuration was validated to confirm successful redirection between Laravel and Cognito Hosted UI. Token generation and validation were also tested to verify secure user authentication and session management.

Testing activities performed:

- User Registration Testing.

- Email Verification Testing.
- Login Testing.
- Logout Testing.
- Token Validation Testing.
- Callback URL Testing.
- Hosted UI Testing.

RESULTS AND OUTPUT

The implementation successfully demonstrated the integration of AWS Cognito with a Laravel application. All major authentication functionalities including user registration, email verification, login, logout, and token generation were tested and validated successfully.

The Hosted UI provided a secure and user-friendly authentication experience. The application was able to redirect users appropriately, validate authentication tokens, and restrict access to protected resources. These results confirm that AWS Cognito can effectively replace traditional authentication mechanisms while providing improved security and scalability. The project was successfully implemented.

- User Pool created successfully.
- App Client configured successfully.
- Hosted UI operational.
- User registration successful.
- Authentication successful.
- Laravel dashboard accessible.
- Logout functionality working.

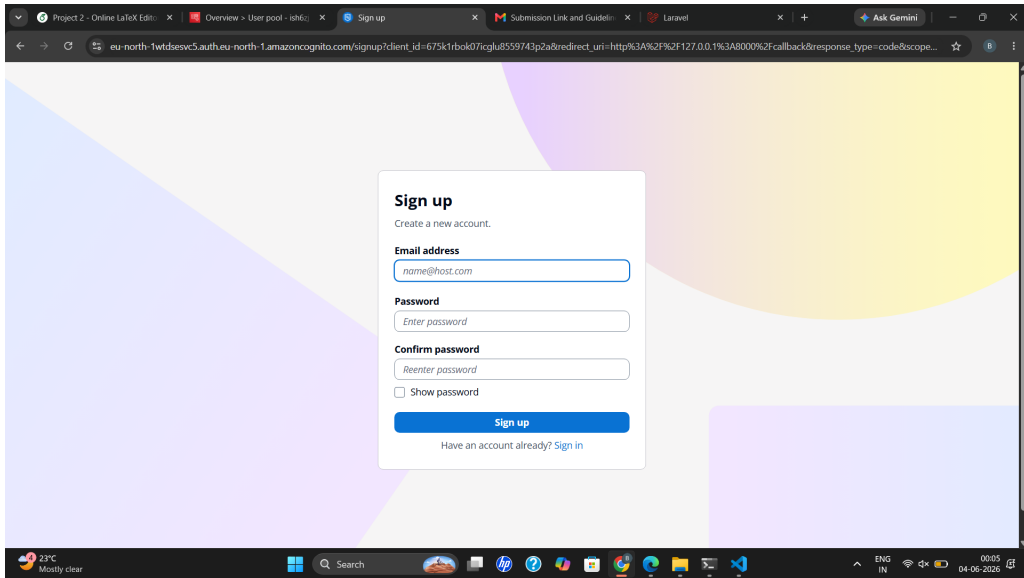


Figure 7: User Registration

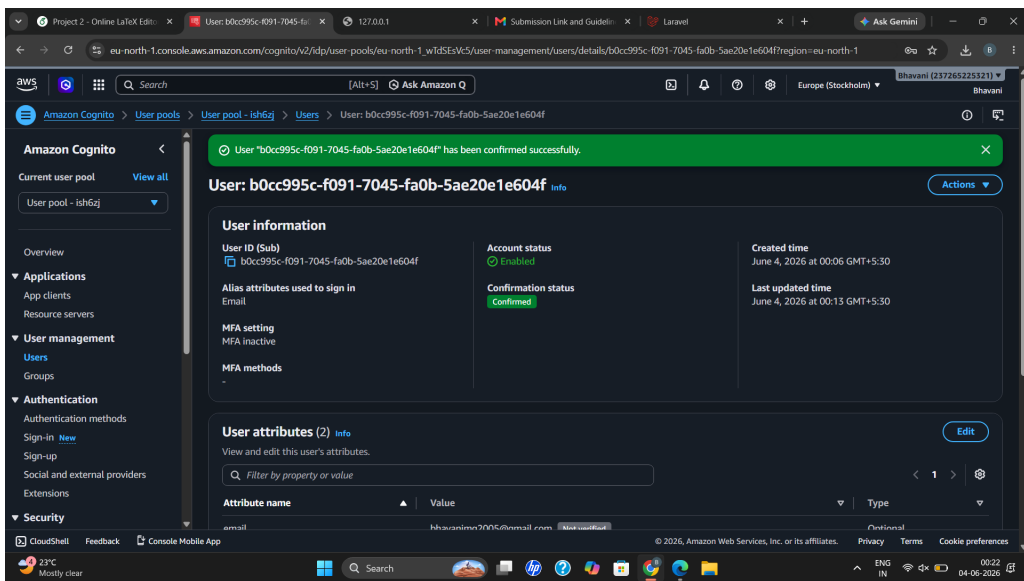


Figure 8: Dashboard

AWS COGNITO SECURITY FEATURES

AWS Cognito provides several security mechanisms that protect user accounts and application resources. Password policies can be configured to enforce strong passwords containing uppercase letters, lowercase letters, numbers, and special characters.

Email verification ensures that only legitimate users can activate accounts. Cognito also supports Multi-Factor Authentication (MFA), which adds an additional security layer by requiring users to verify their identity through a secondary factor.

Authentication tokens generated by Cognito are digitally signed JSON Web Tokens (JWTs). These tokens prevent unauthorized modifications and provide secure identity verification across distributed systems.

All communication between the application and Cognito occurs through HTTPS encryption, ensuring secure transmission of sensitive information across networks.

ADVANTAGES

The implementation of AWS Cognito with Laravel provides several benefits compared to traditional authentication systems. Since authentication is managed by AWS, developers can focus on application development rather than maintaining complex authentication infrastructure. The solution offers enhanced security, scalability, and reliability while reducing operational overhead.

- Secure Authentication using industry-standard protocols such as OAuth 2.0 and OpenID Connect.
- Managed Identity Service that eliminates the need to develop authentication mechanisms from scratch.
- High Availability through AWS cloud infrastructure and distributed services.
- Automatic Scaling to support growing numbers of users without manual intervention.
- Reduced Development Time due to prebuilt authentication and user management features.
- OAuth 2.0 Support for secure authorization and token-based authentication.
- Password Recovery Support through built-in account recovery workflows.
- Email Verification to ensure the authenticity of registered users.
- Improved Security through encrypted communication and JWT-based authentication.

- Easy Integration with web and mobile applications.
- Lower Maintenance Cost as AWS manages authentication infrastructure and updates.
- Enhanced User Experience through Cognito Hosted UI and streamlined login workflows.

CHALLENGES FACED

During the implementation of the project, several technical challenges were encountered. Configuring AWS Cognito and integrating it with Laravel required careful handling of authentication flows and callback URLs.

The OAuth 2.0 authentication process involves multiple steps including authorization code generation, token exchange, and token validation. Understanding and configuring these workflows correctly required extensive testing and debugging.

Challenges Faced:

- Callback URL configuration.
- OAuth configuration issues.
- Token handling and validation.
- Laravel integration challenges.
- SSL configuration.
- Environment variable management.
- Debugging authentication redirects.
- Testing user verification workflows.

FUTURE ENHANCEMENTS

The current implementation provides a secure authentication solution for web applications. However, several advanced features can be incorporated in future versions to improve functionality and user experience.

Future development can focus on strengthening security mechanisms, supporting multiple authentication providers, and extending compatibility across different platforms.

- Multi-Factor Authentication (MFA).
- Social Login Integration using Google and Facebook.
- Mobile App Authentication.
- Role-Based Access Control (RBAC).
- Kubernetes Deployment.
- Single Sign-On (SSO) integration.
- Advanced user analytics and monitoring.
- Biometric authentication support.

CONCLUSION

The User Login System using AWS Cognito and LAMP Stack was successfully implemented. AWS Cognito provided a secure and scalable authentication mechanism while Laravel enabled rapid web application development. The project improved understanding of cloud authentication, OAuth protocols, user management, and deployment methodologies. The solution can be extended for enterprise applications requiring secure and scalable identity management.

LEARNING OUTCOMES

Through this internship project, valuable knowledge was gained in cloud computing, authentication systems, web application security, and deployment methodologies. Practical experience was obtained in configuring AWS services, integrating third-party authentication providers, and implementing OAuth 2.0 authentication workflows.

The project also improved understanding of Laravel development, environment configuration, token-based authentication, and secure application design principles. These skills are highly relevant for modern cloud-native application development.

REFERENCES

1. AWS Cognito Documentation
2. Laravel Documentation
3. AWS EC2 Documentation
4. Apache Documentation
5. PHP Documentation
6. OAuth 2.0 Specification